

Task 1 : French to Tamil

Introduction

French to Tamil 5 word translation internship project involves the design and development of a machine learning–based French to Tamil translation system. The primary objective of the project is to translate French words into Tamil while enforcing a strict constraint that only French words containing exactly five letters are eligible for translation. Any word that does not meet this condition is rejected by the system. This constraint ensures controlled input, reduces ambiguity, and improves translation accuracy.

The project combines concepts from natural language processing, deep learning, and software development. A character-level sequence-to-sequence neural network model is used for translation, and a graphical user interface (GUI) is developed to provide a simple and interactive user experience. The system demonstrates the practical application of machine learning models in real-world language translation tasks.

Background

Machine translation is an important application of artificial intelligence that enables communication between speakers of different languages. Traditional translation systems relied heavily on rule-based or statistical methods. However, recent advancements in deep learning have significantly improved translation quality through neural machine translation. Sequence-to-sequence (Seq2Seq) models, particularly those based on recurrent neural networks such as Long Short-Term Memory (LSTM), are widely used for translation tasks. These models consist of an encoder that processes the input sequence and a decoder that generates the output sequence. In this project, a character-level Seq2Seq LSTM model is employed, which is especially effective for handling short words and learning fine-grained linguistic patterns.

Learning Objectives

The internship was undertaken with the objective of gaining hands-on experience in machine learning and natural language processing. One of the primary learning goals was to understand the working of sequence-to-sequence models and their application in language translation.

Additional objectives included learning data preprocessing techniques, implementing character-level tokenization, training and validating deep learning models, saving and loading trained models, and deploying the model using a graphical user interface. The internship also aimed to improve problem-solving skills and practical understanding of end-to-end machine learning workflows.

Activities and Tasks

The project began with dataset preparation, where a dataset containing French and Tamil word pairs was collected and filtered to include only five-letter French words. The data was preprocessed by adding start and end tokens to the Tamil words to facilitate decoding.

Subsequent tasks included character-level tokenization, sequence padding, and splitting the dataset into training and validation sets. An encoder-decoder LSTM architecture was designed and trained using TensorFlow and Keras. After training, separate encoder and decoder models were created for inference.

Dataset :

	French	Tamil
0	était	இருந்தது
1	faire	செய்ய
2	comme	என
3	votre	உங்கள்
4	cette	இது
...	...	...
982	poète	கவிஞர்
983	admis	ஒப்புக்கொண்டார்
984	drake	டிரேக்
985	lydia	லிட்யா
986	chloe	சோலி

Model :

```
Create a model

Generate Code Markdown

french_vocab_size = len(french_tk.word_index) + 1
tamil_vocab_size = len(tamil_tk.word_index) + 1
latent_dim = 256

encoder_inputs = Input(shape=(max_french_len,))
enc_emb = Embedding(french_vocab_size, 128, mask_zero=True)(encoder_inputs)
_, state_h, state_c = LSTM(latent_dim, return_state=True)(enc_emb)
encoder_states = [state_h, state_c]

decoder_inputs = Input(shape=(max_tamil_len - 1,))
dec_emb_layer = Embedding(tamil_vocab_size, 128, mask_zero=True)
dec_emb = dec_emb_layer(decoder_inputs)

decoder_lstm = LSTM(latent_dim, return_sequences=True, return_state=True)
decoder_outputs, _, _ = decoder_lstm(dec_emb, initial_state=encoder_states)

decoder_dense = Dense(tamil_vocab_size, activation='softmax')
decoder_outputs = decoder_dense(decoder_outputs)

model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
model.summary()
```

Model: "model_8"

Layer (type)	Output Shape	Param #	Connected to
input_10 (InputLayer)	[(None, 5)]	0	[]
input_11 (InputLayer)	[(None, 21)]	0	[]
embedding_5 (Embedding)	(None, 5, 128)	4736	['input_10[0][0]']
embedding_6 (Embedding)	(None, 21, 128)	7168	['input_11[0][0]']
lstm_5 (LSTM)	[(None, 256), (None, 256), (None, 256)]	394240	['embedding_5[0][0]']
lstm_6 (LSTM)	[(None, 21, 256), (None, 256), (None, 256)]	394240	['embedding_6[0][0]', 'lstm_5[0][1]', 'lstm_5[0][2]']
dense_3 (Dense)	(None, 21, 56)	14392	['lstm_6[0][0]']

=====

Total params: 814,776
Trainable params: 814,776
Non-trainable params: 0

=====

Training :

```
model.fit(
    [X_train, decoder_input_train],
    decoder_target_train,
    validation_data=([X_val, decoder_input_val], decoder_target_val),
    batch_size=32,
    epochs=100,
    verbose=2
)
```

Epoch 1/100
25/25 - 14s - loss: 1.5138 - accuracy: 0.1828 - val_loss: 1.3237 - val_accuracy: 0.1770 - 14s/epoch - 568ms/step
Epoch 2/100
25/25 - 1s - loss: 1.2587 - accuracy: 0.2401 - val_loss: 1.2281 - val_accuracy: 0.2891 - 571ms/epoch - 23ms/step
Epoch 3/100
25/25 - 0s - loss: 1.1521 - accuracy: 0.3117 - val_loss: 1.1185 - val_accuracy: 0.3460 - 446ms/epoch - 18ms/step
Epoch 4/100
25/25 - 1s - loss: 1.0518 - accuracy: 0.3594 - val_loss: 1.0280 - val_accuracy: 0.3660 - 569ms/epoch - 23ms/step
Epoch 5/100
25/25 - 0s - loss: 0.9736 - accuracy: 0.3725 - val_loss: 0.9731 - val_accuracy: 0.3698 - 467ms/epoch - 19ms/step
Epoch 6/100
25/25 - 1s - loss: 0.9349 - accuracy: 0.3797 - val_loss: 0.9464 - val_accuracy: 0.3871 - 596ms/epoch - 24ms/step
Epoch 7/100
25/25 - 0s - loss: 0.9074 - accuracy: 0.3977 - val_loss: 0.9200 - val_accuracy: 0.4147 - 460ms/epoch - 18ms/step
Epoch 8/100
25/25 - 0s - loss: 0.8863 - accuracy: 0.4038 - val_loss: 0.9036 - val_accuracy: 0.4277 - 450ms/epoch - 18ms/step
Epoch 9/100
25/25 - 1s - loss: 0.8665 - accuracy: 0.4169 - val_loss: 0.8934 - val_accuracy: 0.4348 - 510ms/epoch - 20ms/step
Epoch 10/100
25/25 - 0s - loss: 0.8496 - accuracy: 0.4308 - val_loss: 0.8707 - val_accuracy: 0.4429 - 492ms/epoch - 20ms/step
Epoch 11/100
25/25 - 0s - loss: 0.8298 - accuracy: 0.4451 - val_loss: 0.8610 - val_accuracy: 0.4537 - 462ms/epoch - 18ms/step
Epoch 12/100
25/25 - 0s - loss: 0.8108 - accuracy: 0.4551 - val_loss: 0.8474 - val_accuracy: 0.4640 - 442ms/epoch - 18ms/step
Epoch 13/100
...
Epoch 99/100
25/25 - 1s - loss: 0.0032 - accuracy: 1.0000 - val_loss: 1.3849 - val_accuracy: 0.4943 - 565ms/epoch - 23ms/step
Epoch 100/100
25/25 - 1s - loss: 0.0031 - accuracy: 1.0000 - val_loss: 1.3917 - val_accuracy: 0.4970 - 596ms/epoch - 24ms/step

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings.

Testing :

```
def translate_french_word(french_word):
    if len(french_word) != 5:
        return "Word must have exactly 5 letters"

    seq = french_tk.texts_to_sequences([french_word])
    seq = pad_sequences(seq, maxlen=max_french_len, padding='post')

    return decode_sequence(seq)

french_input = input("Enter a 5-letter French word: ").strip().lower()
tamil_output = translate_french_word(french_input)

print(f"\nFR French: {french_input}")
print(f"IN Tamil: {tamil_output}")

Enter a 5-letter French word: gamin

FR French: gamin
IN Tamil: குழந்தை
```

Skills and Competencies

This internship significantly enhanced technical skills in Python programming and deep learning. Practical experience was gained in using TensorFlow and Keras for building and training neural networks.

The project also strengthened understanding of natural language processing concepts such as tokenization, sequence modeling, and inference decoding. Additionally, skills in GUI development using Tkinter were developed, enabling integration of machine learning models with user-facing applications.

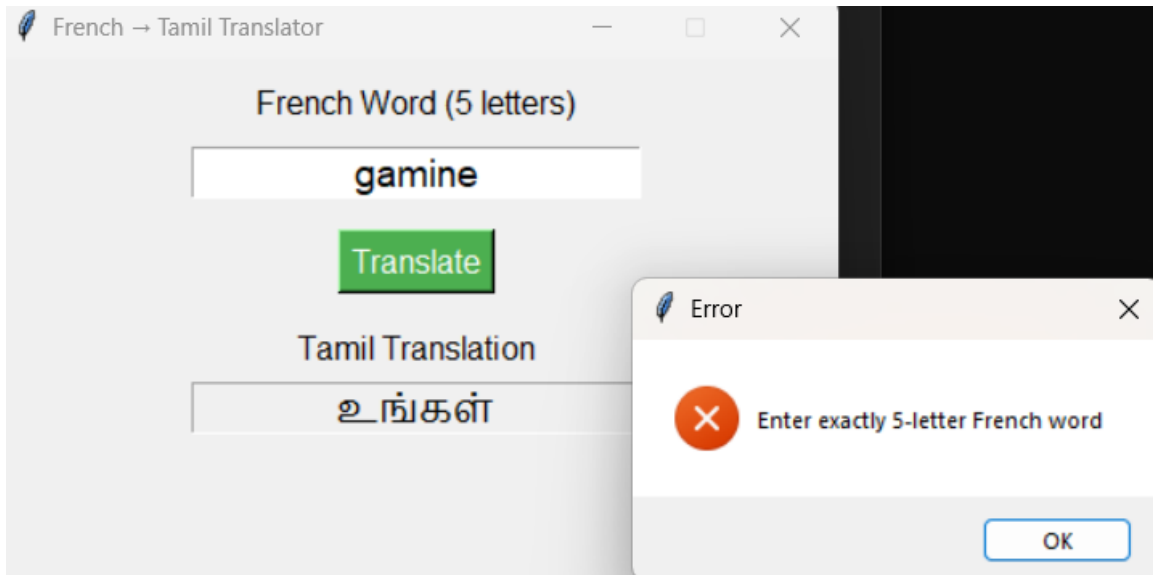
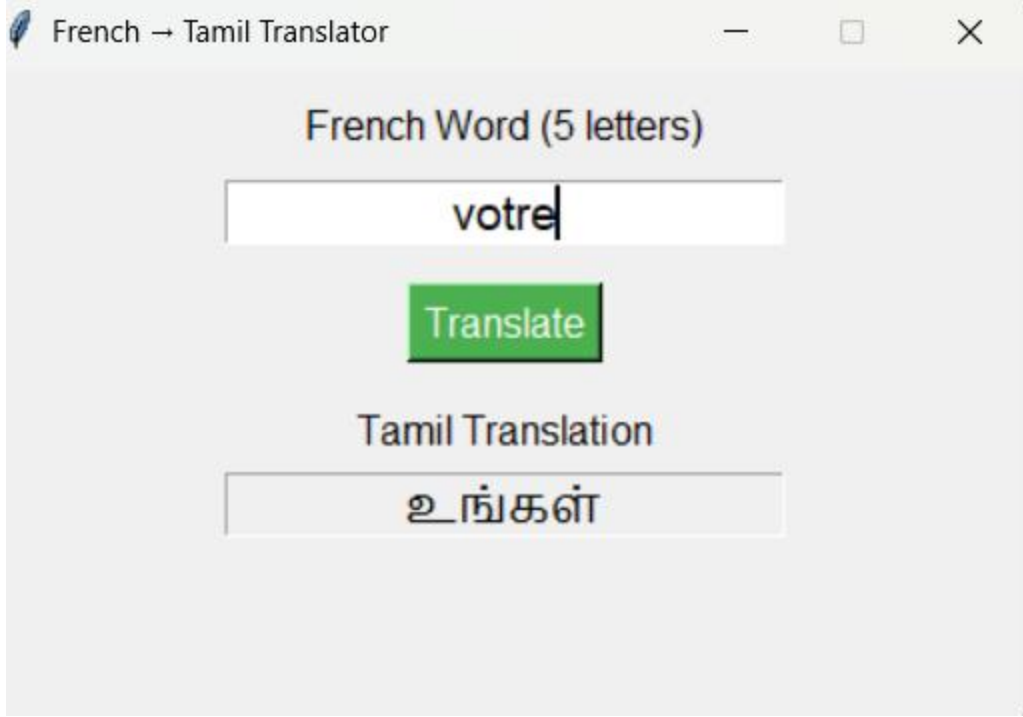
Feedback and Evidence

The developed system was tested using multiple five-letter French words, and the translations were successfully generated in Tamil. The GUI provided immediate feedback and displayed appropriate error messages when invalid input was entered.

The correct functioning of the model and GUI serves as evidence of successful implementation of the project requirements. The project outcomes were reviewed and validated through consistent testing and result verification.

GUI and Testing :





Challenges and Solutions

One of the major challenges faced during the project was implementing the inference mechanism for the sequence-to-sequence model. This was addressed by creating separate encoder and decoder models to handle step-by-step decoding during translation.

Another challenge was enforcing the exact five-letter constraint. This issue was resolved by implementing input validation logic both in the model-level translation function and within the graphical user interface, ensuring robust input handling.

Outcomes and Impact

The final outcome of the internship project is a fully functional French to Tamil translation system with a graphical user interface. The system accurately translates valid inputs and rejects invalid ones, meeting all defined project requirements.

The project had a positive impact on understanding practical machine learning deployment and demonstrated how AI models can be integrated into real-world applications. It also laid a strong foundation for future work in natural language processing.

Conclusion

The internship project successfully achieved its objectives by designing, training, and deploying a constrained French to Tamil machine learning translation system. The integration of a deep learning model with a user-friendly GUI highlights the practical application of artificial intelligence in language translation.

Overall, the project provided valuable hands-on experience in machine learning, enhanced technical competencies, and contributed significantly to professional and academic development.